

BÁO CÁO KẾT QUẢ THỰC TẬP

Tuần 6: Kiến trúc Microservices, Giao thức MQTT và Truyền dữ liệu Thời gian thực với WebSocket

I. LÝ THUYẾT

1. Kiến trúc Microservices và so sánh với Monolithic

- **Kiến trúc Monolithic (Đơn khối):** Toàn bộ mã nguồn, logic nghiệp vụ, giao diện và truy vấn cơ sở dữ liệu được đóng gói trong một ứng dụng duy nhất. Ưu điểm là dễ triển khai ban đầu, nhưng nhược điểm lớn là khó mở rộng độc lập, rủi ro cho toàn bộ hệ thống khi một module gặp sự cố.
- **Kiến trúc Microservices (Vi dịch vụ):** Hệ thống được chia nhỏ thành nhiều dịch vụ độc lập, mỗi dịch vụ đảm nhận một miền nghiệp vụ riêng biệt và giao tiếp với nhau thông qua các giao thức truyền tin. Kiến trúc này làm cho khả năng mở rộng dễ dàng hơn, cô lập lỗi và linh hoạt trong việc lựa chọn công nghệ cho từng dịch vụ.
- **NestJS Microservices:** NestJS cung cấp tầng trừu tượng giúp chuyển đổi ứng dụng sang mô hình Microservice thông qua gói [@nestjs/microservices](#), hỗ trợ các phương thức giao tiếp hướng sự kiện và truy vấn hai chiều (Request-Response).

2. Tổng quan về giao thức MQTT trong IoT

- **MQTT (Message Queuing Telemetry Transport):** Là giao thức truyền tin siêu nhẹ theo mô hình Publish/Subscribe, chạy trên nền TCP/IP.
- **Đặc điểm nổi bật:** Khung bản tin cực nhỏ (Header nhỏ chỉ từ 2 bytes), băng thông thấp, tiết kiệm năng lượng, hoạt động ổn định trên các thiết bị phần cứng hạn chế tài nguyên (ESP32, STM32, Arduino).
- **Cơ chế Publish/Subscribe:**
 - **MQTT Broker:** Trạm trung chuyển tin nhắn trung tâm (Ví dụ: [broker.hivemq.com](#))
 - **Publisher:** Thiết bị phát dữ liệu lên một chủ đề cụ thể (Topic)
 - **Subscriber:** Lắng nghe và nhận dữ liệu từ chủ đề mà mình đăng ký

3. Giao thức WebSocket và sự khác biệt với HTTP

- **So sánh HTTP và WebSocket:**
 - **HTTP:** Là giao thức Một chiều, hoạt động theo mô hình Request/Response. Client luôn phải chủ động gửi request để nhận lại data từ Server.
 - **WebSocket:** Là giao thức Hai chiều, duy trì một kết nối TCP liên tục giữa Client và Server sau một bắt tay kết nối ban đầu.
- **Ứng dụng:** WebSocket giúp Server chủ động đẩy dữ liệu về phía Client ngay khi có sự kiện phát sinh mà không cần Client phải tốn tài nguyên gửi truy vấn liên tục (Polling), thích hợp cho các ứng dụng giám sát cảm biến thời gian thực.

II. NỘI DUNG THỰC HÀNH & CÁC CÂU LỆNH CẤU HÌNH

1. Nội dung thực hành

- **Khởi tạo Hybrid Application trong NestJS:** Cấu hình `src/main.ts` chạy song song cả ứng dụng Web (HTTP/WebSockets) và Microservice lắng nghe bản tin MQTT từ public broker `broker.hivemq.com:1883`.
- **Xây dựng WebSocket Gateway (TelemetryGateway):** Triển khai giao thức Socket.IO để duy trì kết nối thời gian thực với các Client giám sát (Postman/Frontend).
- **Xử lý Luồng dữ liệu Cảm biến (Upstream):** Đăng ký lắng nghe Topic `esp32/sensor_data` thông qua decorator `@MessagePattern()`. Khi nhận được gói tin dữ liệu chuẩn JSON từ thiết bị IoT, hệ thống tự động đẩy gói tin này qua WebSocket tới Client.
- **Xử lý Luồng điều khiển Thiết bị (Downstream):** Xây dựng REST API `POST /device/command` truyền lệnh tới `ClientsModule` (MQTT Transport) để gửi ngược bản tin điều khiển tới thiết bị thông qua Topic `esp32/commands`.

2. Các câu lệnh cấu hình và cài đặt thư viện

Mở Terminal tại thư mục gốc dự án và thực thi lệnh cài đặt các gói phụ thuộc cho Tuần 6:

- **Cài đặt thư viện Microservices và bộ chuyển đổi kết nối MQTT**

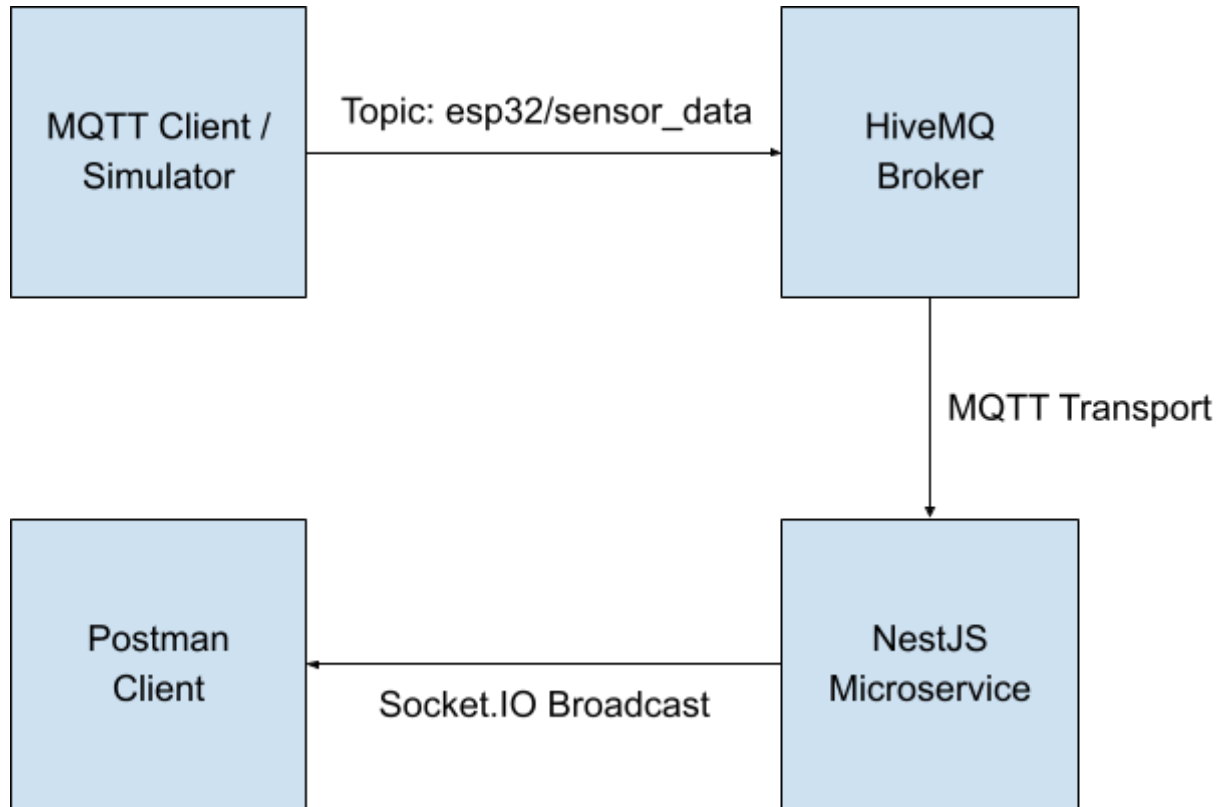
```
npm install @nestjs/microservices mqtt
```

- **Cài đặt bộ công cụ WebSocket và Socket.IO cho NestJS**

```
npm install @nestjs/websockets @nestjs/platform-socket.io socket.io
```

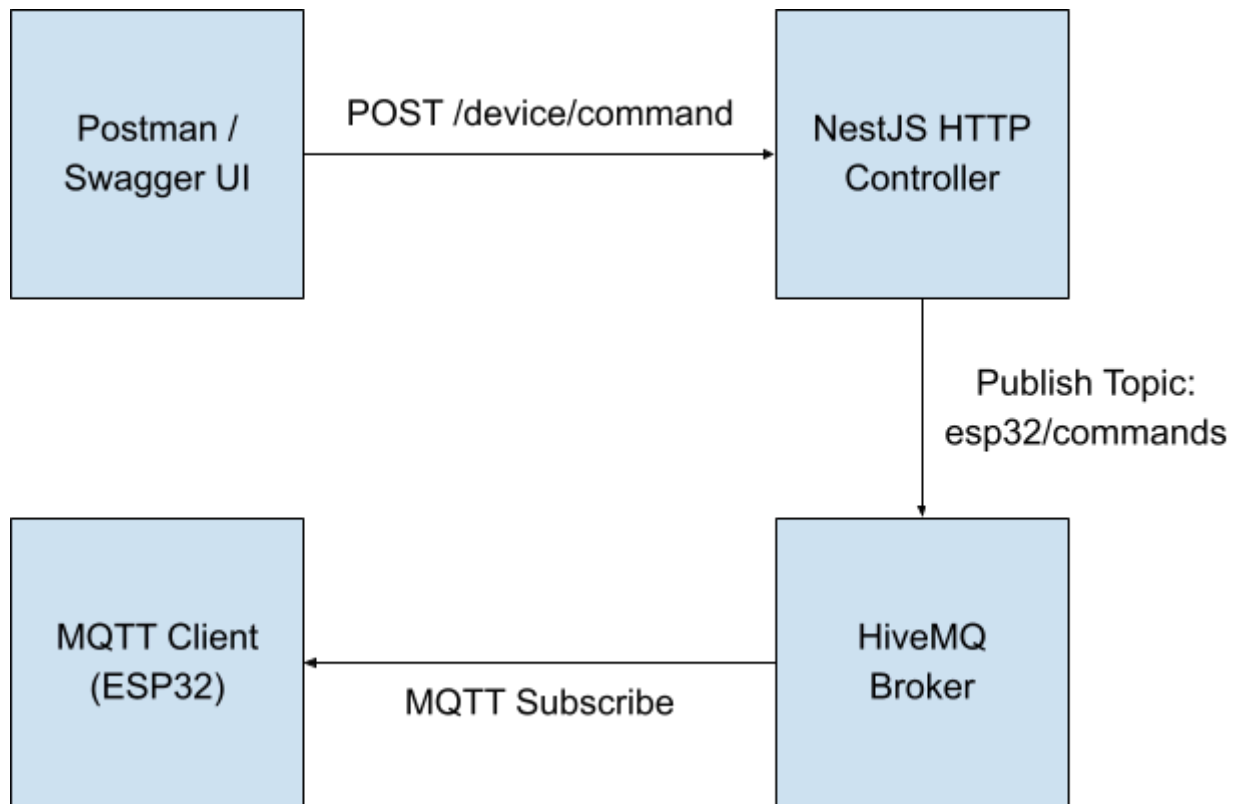
III. LƯUỒNG THỰC HIỆN CỦA HỆ THỐNG

1. Luồng 1: Thu thập và Hiển thị dữ liệu Cảm biến Thời gian thực (Upstream)



- **Bước 1:** Thiết bị IoT (ESP32 / MQTXX) đóng gói dữ liệu cảm biến dạng JSON và Publish lên Topic `esp32/sensor_data` trên Broker `broker.hivemq.com:1883`.
- **Bước 2:** NestJS Microservice đóng vai trò Subscriber tiếp nhận dữ liệu và chuyển cho `TelemetryMqttController`.
- **Bước 3:** Controller gọi hàm `broadcastTelemetry()` của `TelemetryGateway` để phát dữ liệu tới tất cả các Client đang mở kết nối WebSocket.

2. Luồng 2: Phát lệnh Điều khiển Thiết bị từ Server (Downstream)

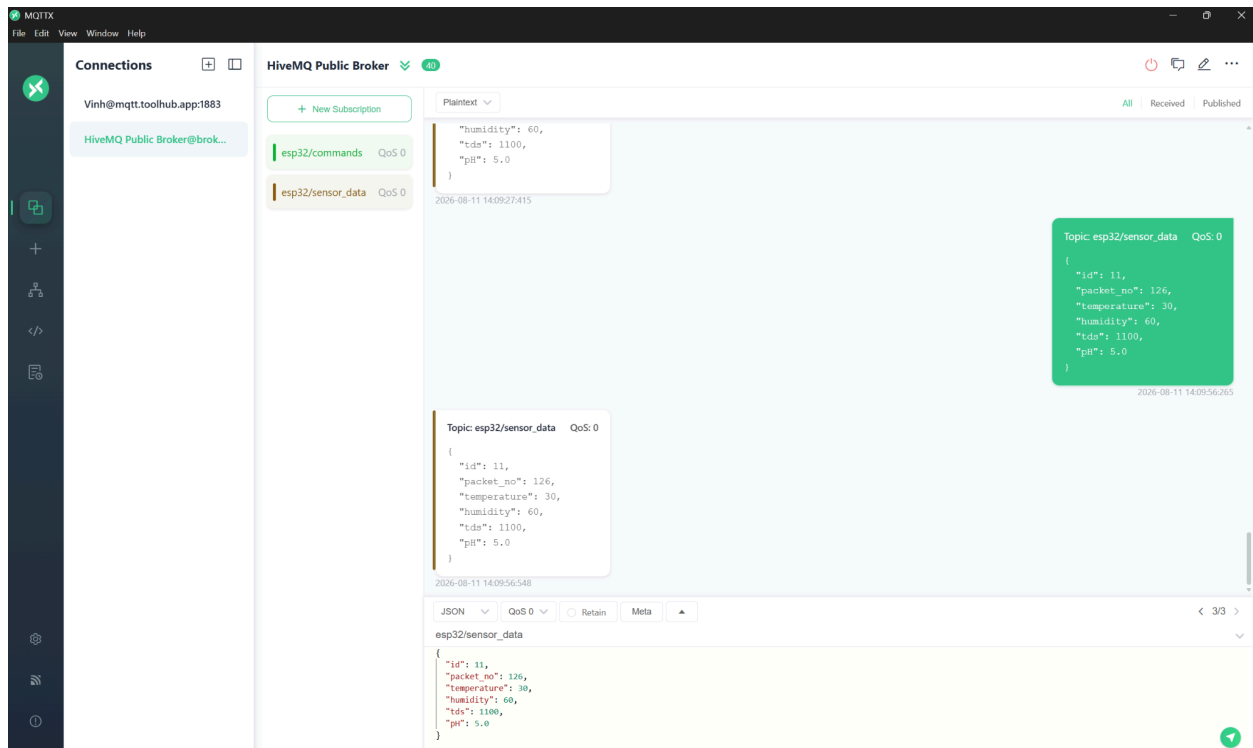


- **Bước 1:** Người dùng gửi yêu cầu HTTP REST API tới endpoint **POST /device/command**.
- **Bước 2:** **TelemetryHttpController** nhận dữ liệu và dùng **ClientProxy** phát bản tin tới Topic **esp32/commands** trên MQTT Broker.
- **Bước 3:** Thiết bị IoT (ESP32) đã Subscribe topic này lập tức nhận được lệnh và thi hành thao tác phần cứng.

IV. KẾT QUẢ KIỂM THỬ HỆ THỐNG

1. Kiểm thử Luồng 1: Nhận dữ liệu Cảm biến qua MQTT/Simulator và Đẩy lên WebSocket (Upstream)

- Trường hợp 1A: Phát dữ liệu thủ công từ MQTT Client (MQTTX)
 - Thông số cấu hình:
 - Broker: `broker.hivemq.com:1883`
 - Topic: `esp32/sensor_data`
 - Kết quả:
 - Postman (Socket.IO `ws://localhost:3000`): Postman duy trì kết nối WebSocket liên tục, lắng nghe event `sensor_data` và nhận được đúng bản tin JSON từ Server chuyển tiếp theo thời gian thực.
 - MQTTX: Hiển thị bản tin màu xanh (Publish) gửi đi thành công lên topic `esp32/sensor_data`, đồng thời nhận phản hồi hiển thị bản tin (Subscribe) từ HiveMQ Broker xác nhận dữ liệu đã qua Broker trôi chảy.



RR Hoang Duong Minh's Workspace

Web Socket: My App - Web Socket

ws://localhost:3000

Message

1 Compose message

+ Arg Text

Ack Event name, defaults to "message" Send

Response

Listening to 1 events Connected Save Response

sensor_data {"id":11,"packet_no":126,"temperature":30,"humidity":60,"tds":1100,"ph":5}

JSON

```
1 {
2   "id": 11,
3   "packet_no": 126,
4   "temperature": 30,
5   "humidity": 60,
6   "tds": 1100,
7   "ph": 5
8 }
```

Listening to sensor_data 14:30:42.168

Connected to ws://localhost:3000 14:30:42.155

```
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [NestMicroservice] Nest microservice successfully started +651ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RoutesResolver] AppController {}: +19ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/hello, GET} route +2ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RoutesResolver] AuthController {/auth}: +0ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/auth/register, POST} route +1ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/auth/login, POST} route +0ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RoutesResolver] UserController {/users}: +0ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/users, GET} route +1ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/users, PATCH} route +1ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/users/:id, GET} route +0ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +2ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RoutesResolver] TelemetryMqttController {}: +0ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RoutesResolver] TelemetryHttpController {/device}: +1ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [RouterExplorer] Mapped {/device/command, POST} route +3ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [NestApplication] Nest application successfully started +87ms
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [Bootstrap] HTTP & WebSockets running on: http://localhost:3000
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [Bootstrap] MQTT Microservice connected to: mqtt://broker.hivemq.com:1883
[Nest] 25972 - 08/11/2026, 2:57:10 PM LOG [Bootstrap] Swagger UI documentation available at: http://localhost:3000/api
[Nest] 25972 - 08/11/2026, 2:57:14 PM LOG [TelemetryGateway] Client connected: _U7PaPfntW1RpwIgAAAB
[Nest] 25972 - 08/11/2026, 2:57:19 PM LOG [TelemetryMqttController] Received MQTT sensor packet #126 from Device ID 11
```

- **Trường hợp 1B: Phát dữ liệu ngẫu nhiên tự động từ Server (Telemetry Simulator)**
 - **Cơ chế:** `TelemetrySimulatorService` chạy `@Interval(1000)` tự động tạo và đẩy gói tin cảm biến biến thiên liên tục mỗi 1 giây.
 - **Kết quả:** Trên Terminal xuất hiện log `[TelemetrySimulator] [Interval] Broadcasted real-time packet #X`, đồng thời dữ liệu trên Postman tự động cập nhật liên tục theo thời gian thực.

```
[Nest] 6356 - 08/11/2026, 2:56:16 PM LOG [NestApplication] Nest application successfully started +95ms
[Nest] 6356 - 08/11/2026, 2:56:16 PM LOG [Bootstrap] HTTP & WebSockets running on: http://localhost:3000
[Nest] 6356 - 08/11/2026, 2:56:16 PM LOG [Bootstrap] MQTT Microservice connected to: mqtt://broker.hivemq.com:1883
[Nest] 6356 - 08/11/2026, 2:56:16 PM LOG [Bootstrap] Swagger UI documentation available at: http://localhost:3000/api
[Nest] 6356 - 08/11/2026, 2:56:17 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #1
[Nest] 6356 - 08/11/2026, 2:56:17 PM LOG [TelemetryGateway] Client connected: pnln9vIErdkZW8OZAAAB
[Nest] 6356 - 08/11/2026, 2:56:18 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #2
[Nest] 6356 - 08/11/2026, 2:56:19 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #3
[Nest] 6356 - 08/11/2026, 2:56:20 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #4
[Nest] 6356 - 08/11/2026, 2:56:21 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #5
[Nest] 6356 - 08/11/2026, 2:56:22 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #6
[Nest] 6356 - 08/11/2026, 2:56:23 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #7
[Nest] 6356 - 08/11/2026, 2:56:24 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #8
[Nest] 6356 - 08/11/2026, 2:56:25 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #9
[Nest] 6356 - 08/11/2026, 2:56:26 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #10
[Nest] 6356 - 08/11/2026, 2:56:27 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #11
[Nest] 6356 - 08/11/2026, 2:56:28 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #12
[Nest] 6356 - 08/11/2026, 2:56:29 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #13
[Nest] 6356 - 08/11/2026, 2:56:30 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #14
[Nest] 6356 - 08/11/2026, 2:56:31 PM LOG [TelemetrySimulator] [Interval] Broadcasted real-time packet #15
```

The screenshot shows the Postman interface for a WebSocket connection. The left sidebar lists various collections, with 'My App - Web Socket' selected. The main panel shows the 'Message' tab for the connection to `ws://localhost:3000`. Below the message input area, the 'Response' tab displays a list of received messages. Each message is a JSON object representing sensor data. The status bar at the bottom indicates 'Listening to 1 events' and 'Connected'.

Message	Time
<code>{ "id": 11, "packet_no": 85, "temperature": 30.4, "humidity": 71.4, "tds": 838, "pH": 7.3 }</code>	14:55:00.594
<code>{ "id": 11, "packet_no": 84, "temperature": 27.2, "humidity": 45.9, "tds": 1014, "pH": 5.9 }</code>	14:54:59.589
<code>{ "id": 11, "packet_no": 83, "temperature": 21, "humidity": 61.9, "tds": 1291, "pH": 6.1 }</code>	14:54:58.585
<code>{ "id": 11, "packet_no": 82, "temperature": 24.5, "humidity": 41.6, "tds": 756, "pH": 8.1 }</code>	14:54:57.586
<code>{ "id": 11, "packet_no": 81, "temperature": 24.8, "humidity": 64.4, "tds": 539, "pH": 5.9 }</code>	14:54:56.585
<code>{ "id": 11, "packet_no": 80, "temperature": 30.7, "humidity": 59.6, "tds": 669, "pH": 8.5 }</code>	14:54:55.571
<code>{ "id": 11, "packet_no": 79, "temperature": 30.6, "humidity": 58.1, "tds": 1480, "pH": 6.8 }</code>	14:54:54.568
<code>{ "id": 11, "packet_no": 78, "temperature": 31.6, "humidity": 71.1, "tds": 518, "pH": 6.7 }</code>	14:54:53.557
<code>{ "id": 11, "packet_no": 77, "temperature": 31.6, "humidity": 44.3, "tds": 890, "pH": 6.3 }</code>	14:54:52.548
<code>{ "id": 11, "packet_no": 76, "temperature": 21.9, "humidity": 53.9, "tds": 1004, "pH": 7.3 }</code>	14:54:51.541
<code>{ "id": 11, "packet_no": 75, "temperature": 27.4, "humidity": 59.4, "tds": 1421, "pH": 6.7 }</code>	14:54:50.532

2. Kiểm thử Luồng 2: Phát lệnh Điều khiển qua REST API lên MQTT (Downstream)

- Thao tác gửi request qua Postman / Swagger UI:
 - Endpoint: **POST** `http://localhost:3000/device/command`
 - Header: **Content-Type**: `application/json`
- Kết quả nhận được trên MQTT Client (MQTTX): Client (định danh ESP32) đang Subscribe topic `esp32/commands` nhận được bản tin điều khiển đẩy xuống từ Server

